

**Assignment**  
**Course Code: PMSCS 670**  
**Course Name: Software Testing**

1. Write short notes on the following (Any four)
  - (a) Define Software. Differentiate between Software Testing and Software Quality Assurance.
  - (b) Differentiate between fault, error, and failure with appropriate examples and code (if necessary).
  - (c) Explain five different levels of thinking maturity about software testing.
  - (d) Differentiate between Software verification and software validation.
  - (e) Draw the block diagram of a software testing strategy and explain it briefly.
  - (f) Explain 7 principles of software testing.
  - (g) Draw the block diagram of RIPR, V-Model & MDTD and explain them briefly.
  - (h) Explain the V-Model with the necessary diagram.
  - (i) Differentiate between smoke testing, sanity testing and regression testing.
2. Write short note on the followings:
  - i) Alpha testing and Beta testing.
  - ii) Load testing and stress testing.
  - iii) Black box testing and white box testing
  - iv) Differentiate between Functional and non-functional testing.
  - v) Differentiate between Monkey testing and Gorilla testing.
  - vi) Functional and non-functional testing.
3. Define:
  - i) Coverage criterion
  - ii) Test criterion,
  - iii) Test requirements with an appropriate example.
  - iv) Node Coverage
  - v) Edge Coverage
  - vi) Edge Pair Coverage
  - vii) Complete path coverage
4. Explain the basic architecture of a control flow graph. Define simple path and prime path.
5. a) 

```
year = ORIGINYEAR; /* = 1980 */
while (days > 365)
{
    if (IsLeapYear(year))
    {
        if (days > 366)
        {
            days -= 366;
            year += 1;
        }
    }
    else
    {
        days -= 365;
        year += 1;
    }
}
```

There is a bug in the following code. Find and fix the bug also state the input for which the output of the given program will be incorrect.

b)

```

public static int numZero(int[] x)
{
    // Effects: if x == null, throw NullPointerException
    //           else return the number of occurrences of 0 in x
    int count = 0; // line1
    for (int i=1; i<x.length; i++) { // line2
        if (x[i] == 0) // line3
            count++; // line4
    }
    return count; // line5
}

```

$TR = \{NPE-B1, B1, !B1, B2, !B2\}$

Generate test cases for individual test requirements and explain them briefly.

6. Generate CFG for the following code snippets.

a)

```

public static void computeStats (int [ ] numbers)
{
    int length = numbers.length;
    double med, var, sd, mean, sum, varsum;
    sum = 0.0;
    for (int i = 0; i < length; i++)
    {
        sum += numbers [ i ];
    }
    med    = numbers [ length / 2 ];
    mean = sum / (double) length;
    varsum = 0.0;
    int i = 0;
    while(i < length)
    {
        varsum = varsum + ((numbers [i] - mean)*(numbers [i] - mean));
        i++;
    }
    var = varsum / ( length - 1 );
    sd  = Math.sqrt ( var );
    System.out.println ("variance:      " + var);
    System.out.println ("standard deviation:  " + sd);
}

```

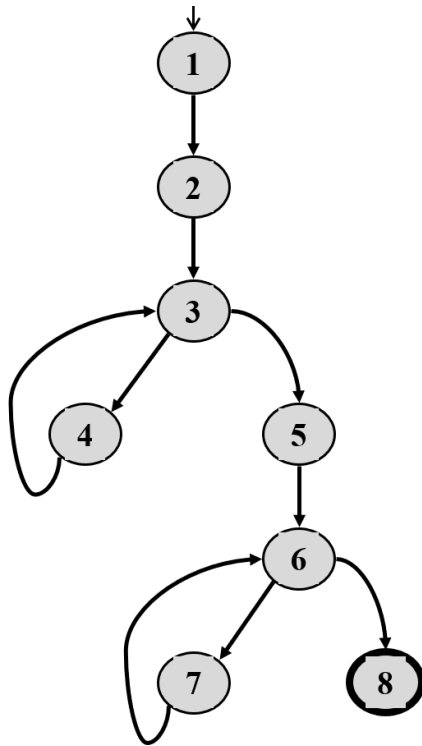
b)

```

try
{
    s = br.readLine();
    if (s.length() > 96)
        throw new Exception
            ("too long");
    if (s.length() == 0)
        throw new Exception
            ("too short");
} (catch IOException e) {
    e.printStackTrace();
} (catch Exception e) {
    e.getMessage();
}
return (s);

```

7.



Determine all the prime path for the above CFG.

8.

Generate CFG for the following code snippet.

5a (i)

```

int i,j=100;
for(i=1;i<=j;i++)
{x = 0;
while (x < y)
{
y = f (x, y);
if (y == 0)
{
break;
} else if (y < 0)
{
y = y*2;
continue;
}
x = x + 1;
}
print (y);
}

```

5a (ii)

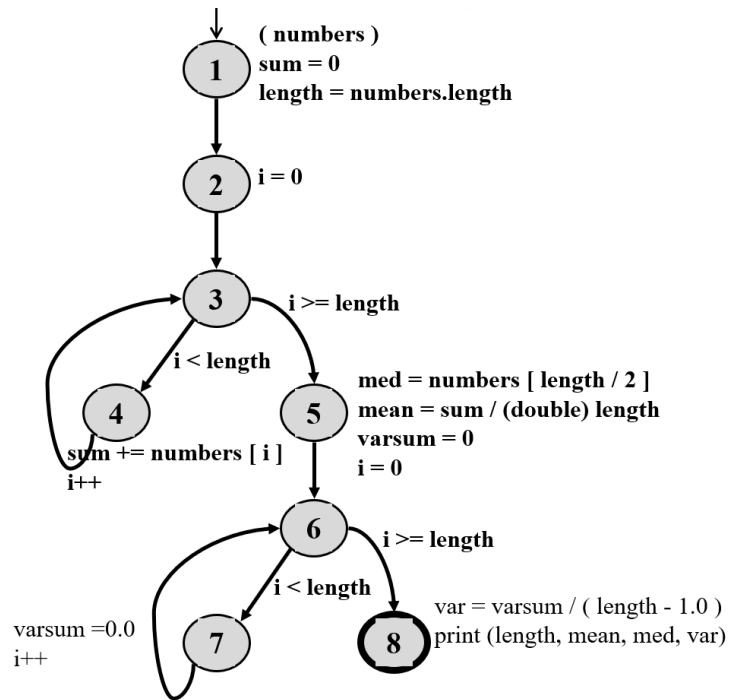
```

read ( c ) ;
switch ( c )
{
case 'N':
z = 25;
break;
case 'Y':
x = 50;
default:
x = 0;
break;
}
print (x);

```

9. Determine the def & use for the following code snippet / CFG.

```
public static void computeStats (int [ ]
numbers)
{
    int length = numbers.length;
    double med, var, sd, mean, sum,
varsum;
    sum = 0.0;
    for (int i = 0; i < length; i++)
    {
        sum += numbers [ i ];
    }
    med = numbers [ length / 2 ];
    mean = sum / (double) length;
    varsum = 0.0;
    for (int i = 0; i < length; i++)
    {
        varsum = varsum + ((numbers [ i ]
- mean) * (numbers [ i ] - mean));
    }
    var = varsum / ( length - 1 );
    System.out.println ("length: " +
length);
    System.out.println ("mean: " + mean);
    System.out.println ("median: " + med);
    System.out.println ("variance: " + var);
}
```



- 10) **public enum** Triangle {  
 SCALENE, ISOSCELES, EQUILATERAL, INVALID  
**}**  
**public class** TriangleType{  
 /\*\*  
 \* @param s1, s2, s3: sides of the putative triangle  
 \* @return enum describing type of triangle  
 \*/  
**public static** Triangle triangle (int s1, int s2, int s3)  
 {  
 // Reject non-positive sides  
**if** (s1 <= 0 || s2 <= 0 || s3 <= 0) **return** (Triangle.INVALID);  
 // Check triangle inequality  
**if** (s1+s2 <= s3 || s2+s3 <= s1 || s1+s3 <= s2) **return** (Triangle.INVALID);  
 // Identify equilateral triangles  
**if** ((s1 == s2) && (s2 == s3)) **return** Triangle.EQUILATERAL;  
 // Identify isosceles triangles  
**if** ((s1 == s2) || (s2 == s3) || (s1 == s3)) **return** Triangle.ISOSCELES;  
**return** (Triangle.SCALENE);  
 }  
**}**

Determine the input domain model for the above code.

b)

| Characteristic | $b_1$ | $b_2$ | $b_3$ | $b_4$ |
|----------------|-------|-------|-------|-------|
| A              | A1    | A2    | A3    | A4    |
| B              | B1    | B2    | B3    | B4    |
| C              | C1    | C2    | C3    | C4    |

- c) **How many test cases are required to satisfy:**
- i) **AcoC**
  - ii) **pair-wise coverage criterion?**
  - iii) **ECC**
  - iv) **T-Wise coverage (consider T=3).**
  - v) **Base Choice coverage considering :**
    - a. **A1, B2, C3 as base .**
    - b. **Bases are: A1, B2, C1 and A1, B1, C3.**
- d)
- i) **Distinguish between interface based approach and functionality based approach.**
  - ii) **Dscribe different steps for modeling the input domain.**
  - iii) **Draw block diagram for ISP Coverage Criteria Subsumption**